A Benchmark Construction and Chunking Algorithms

This appendix summarizes three procedures used for benchmark construction and bounded-context migration: (i) relevance scoring, (ii) constraint-aware file selection, and (iii) PHP-aware smart chunking.

A.1 Benchmark construction

Relevance scoring

We score candidate files using pinned-contract trigger activity and migration-relevance signals.

Algorithm A1. File scoring for migration relevance under the pinned contract.

Require: file with line count LOC, version-change indicator V (`php_version_changes`), and triggered rules R
Ensure: relevance score s

$s \leftarrow 10 \cdot |R| + 5 \cdot V$
if $100 \leq LOC \leq 2000$ **then**
    $s \leftarrow s + 20$
else if $LOC > 2000$ **then**
    $s \leftarrow s - 0.01 \cdot (LOC - 2000)$
end if
$s \leftarrow s + 8 \cdot |\{\text{PHP rule families represented in } R\}|$
$s \leftarrow s + \textsc{ImportantRuleBonus}(R)$
return s

Constraint-aware selection

We select 100 files while enforcing complete rule coverage as a hard constraint.

Algorithm A2. Constraint-aware selection of 100 files with full rule coverage.

Require: summary table, rule metadata, size bins, and target counts T
Ensure: selected file set S

partition files into size categories
compute a relevance score for each file using Algorithm A1
for all size categories c **do**
    add the top $T[c]$ files by score to S
end for
$S \leftarrow \textsc{EnsureRuleCoverage}(S)$ ▷ May add files outside bin quotas; coverage is the hard constraint
if $|S| > 100$ **then**
    remove lowest-scoring files while preserving full rule coverage
else if $|S| < 100$ **then**
    add highest-scoring remaining files until $|S| = 100$
end if
return S

A.2 PHP-aware chunking for bounded-context migration

Smart chunking

We split long PHP files into bounded-context segments while attempting to avoid splitting within function bodies under a fixed chunking policy.

Algorithm A3. PHP-aware smart chunking.

Require: $code$ (PHP source string), $chunk_size$ (target lines), $MAX_EXTENSION$ (maximum extension in lines)
Ensure: $chunks$ (list of records $\{start_line, end_line, actual_size, total_lines, code\}$)

```
lines ← SPLIT(code, "\n")
total ← |lines|
if total ≤ chunk_size then
    return [{1, total, total, total, code}]
end if
boundaries ← FINDFUNCTIONBOUNDARIES(code)    ▷ Tokenizer-guided detection via token_get_all when available; regex fallback; end lines approximated by brace matching
chunks ← [ ]
current ← 0
while current < total do
    initial_end ← min(current + chunk_size − 1, total − 1)
    final_end ← initial_end
    relevant ← {f ∈ boundaries | (current ≤ f.start ≤ initial_end) ∨ (f.start < current ∧ f.end ≥ current)}
    for all f ∈ relevant do
        if f.end ≤ initial_end + MAX_EXTENSION then
            final_end ← max(final_end, f.end)
        end if
    end for
    chunk_code ← JOIN(lines[current..final_end], "\n")
    append {current + 1, final_end + 1, (final_end − current + 1), total, chunk_code} to chunks
    current ← final_end + 1
end while
return chunks
```

B Prompt Templates

We use a shared prompting scheme across models, with separate variants for whole-file migration and chunk-based migration. In both templates, `{code}` is replaced with the input PHP source.

B.1 Whole-file prompt

Whole-file prompt

```
You are a senior PHP developer with expertise in legacy code modernization.
Your task is to migrate this legacy PHP code to PHP 8.3 using modern syntax and
↪  features while preserving functional equivalence.

Please migrate the following PHP code to PHP 8.3:

{code}

Your response must follow this EXACT format:

// MIGRATION_START
[your migrated PHP code here]
// MIGRATION_END

CRITICAL FORMATTING REQUIREMENT:
- Place the MIGRATION_START marker BEFORE the opening <?php tag
- Place the MIGRATION_END marker AFTER the PHP code
- Do NOT place these markers inside the PHP code itself

Provide only the migrated PHP code with the markers placed correctly outside the PHP
↪  code block. Do not include any additional commentary.
```

B.2 Chunk-segment prompt

The placeholders `{filename}`, `{start_line}`, `{end_line}`, `{total_lines}`, `{chunk_number}`, and `{total_chunks}` are populated from the chunk metadata.

Chunk-segment prompt

```
You are a senior PHP developer with expertise in legacy code modernization.
Your task is to migrate this PARTIAL SEGMENT of a larger PHP file to PHP 8.3 using
↪  modern syntax and features.

CONTEXT:
- Original file: {filename}
- Processing lines: {start_line} to {end_line} (of {total_lines} total lines)
- This is chunk {chunk_number} of {total_chunks}

CRITICAL INSTRUCTIONS FOR PARTIAL CODE SEGMENTS:
1. This is ONLY a SEGMENT of a larger file. DO NOT try to complete it.
2. Keep the original opening <?php tag if present. If the segment does not start
↪  with <?php, DO NOT add one.
3. Keep the original closing ?> tag if present. If the segment does not end with ?>,
↪  DO NOT add one.
4. Keep the original closing braces }} if present. DO NOT add any closing braces
↪  that are not in the original segment.
5. Keep the original opening braces {{ if present. DO NOT add any opening braces
↪  that are not in the original segment.
6. DO NOT try to complete class definitions, function definitions, or other code
↪  structures.
7. Preserve the EXACT start and end boundaries of the provided code segment.

WARNING: Adding extra braces that are not in the original segment, or completing
↪  incomplete code structures, will break the reconstruction process.

Please modernize ONLY the following PHP code segment to PHP 8.3:

{code}
```

```
Your response must follow this EXACT format:

// MIGRATION_START
[your modernized code segment here - exactly as provided, with no additions]
// MIGRATION_END

CRITICAL FORMATTING REQUIREMENT:
- Place the MIGRATION_START marker BEFORE the code segment
- Place the MIGRATION_END marker AFTER the code segment
- Do NOT place these markers inside the code itself

Migrate only the provided code segment. Do not add missing parts or attempt to
↪  complete incomplete structures.
```

C Loadability Tripwire Slice Specification

This appendix documents the *isolated, stubbed loadability tripwire* used on an execution slice to surface harness-visible load-time failures after migration. It specifies the execution environment, harness bootstrap, slice accounting, outcome taxonomy, and diagnostic scope.

C.1 Scope and interpretation

This signal is *harness-dependent*; it is neither an integration test nor an end-to-end correctness guarantee. We interpret it primarily as a *regression tripwire*, focusing on *New failure* outcomes (Orig. Pass → Migr. Fail). Unless otherwise stated, all percentages are computed over the full slice size $N_{\mathrm{rt}} = 55$; no files are excluded. Accordingly, this tier is interpreted as a *within-harness regression detector* for newly introduced load-time failures under identical conditions.

C.2 Regression definition and harness-scoped interpretation

A **regression** is defined as a file that loads successfully in the original snapshot (under PHP 8.3.22 with stubs) but fails to load after migration. These are harness-visible include-time failures: parse errors, fatal errors, uncaught exceptions, and missing-symbol errors that prevent `require_once` from completing.

Observed regression counts across the slice ($N_{\mathrm{rt}} = 55$):

Model	Regressions	Regression Rate
Rector Baseline	7	12.7%
Claude Sonnet 4	8	14.5%
GPT-5 Codex	9	16.4%
Llama 3.3 70B	12	21.8%
Gemini 2.5 Pro	12	21.8%
Gemini 2.5 Flash	18	32.7%

Interpretation scope.

These failures are *harness-visible include-time defects*: the file cannot be included without a fatal condition under the tripwire harness. They are detected under isolated stub conditions rather than full application bootstrap. Accordingly, this harness does not validate behavioral correctness, API-level correctness, or end-to-end functionality; it detects only include-time failures. A file that passes this tripwire may still contain behavioral regressions, and a file that fails here may still succeed under full WordPress initialization. We therefore interpret

New failure counts as a *lower bound* on migration breakage under this harness. They capture include-time breaks (e.g., syntax errors, missing symbols, invalid includes, or signature changes that trigger immediate fatals) that static contract discharge may miss, but they do not capture semantic or integration-level faults.

C.3 Execution environment

All tripwire checks are performed in a fixed environment for repeatability.

Execution environment: All code, both original and migrated, is executed under PHP 8.3.22. When we refer to the "original snapshot," we mean the legacy WordPress 4.0 source snapshot, not the interpreter version used for execution.

Table 1 Execution Environment Specification (Loadability Tripwire)

Component	Specification
PHP Version	8.3.22 (cli)
Operating System	Windows 11 (x64)
Memory Limit	512 MB
Error Reporting	`E_ALL`

Note. We use a single fixed OS and interpreter configuration for repeatability. This tier is interpreted only through regression direction (Orig. Pass → Migr. Fail) under identical harness conditions, not as a portable runtime-correctness metric. Because include-time behavior can be OS- and configuration-sensitive (e.g., filesystem case sensitivity, path handling, or extension availability), cross-environment generalization is out of scope for this signal.

C.4 Execution slice definition ($N_{\mathrm{rt}} = 55$)

We evaluate a fixed execution slice of size $N_{\mathrm{rt}} = 55$ benchmark files. Each file is executed in isolation in a fresh PHP process under a minimal WordPress-core stub environment and is then loaded via `require_once`. We record **loadability outcomes** (load success vs. fatal error/exception).

The tripwire checks only whether each file loads without fatal errors. It does not validate behavioral correctness, API contracts, or end-to-end functionality. Some files may fail to load even in the original snapshot because of isolated execution with stubs (e.g., external dependencies, cross-file coupling, or missing runtime initialization). **All 55 files are retained in the slice**, and such cases are represented using the outcome taxonomy in Section C.7 rather than excluded.

C.5 Harness bootstrap

C.5.1 File loading procedure

Files are loaded using the following procedure, implemented in `run_one.php`:

1. Load WordPress core stubs (`stubs/wp_core.php`)
2. Translation-entry stubs (`stubs/entry.php`) are conditionally loaded by `wp_core.php`
3. Include the file under test via `require_once`
4. Record load success or fatal error

Table 2 Stubbed WordPress APIs and Globals (Loadability Tripwire Harness)

Category	Stubbed Elements
Global Variables	`$wpdb`, `$wp_filter`, `$wp_actions`, `$wp_current_filter`, `$wp_query`, `$post`, `$wp_rewrite`
I18n Functions	`__()`, `_e()`, `_x()`, `_n()`, `_nx()`, `esc_html__()`, `esc_attr__()`
Hook System	`add_action()`, `remove_action()`, `do_action()`, `add_filter()`, `remove_filter()`, `apply_filters()`, `has_filter()`, `has_action()`, `did_action()`, `current_filter()`
Sanitization	`esc_url()`, `esc_js()`, `esc_sql()`, `wp_kses_post()`
Utilities	`wp_parse_args()`, `absint()`, `wp_unslash()`, `wp_slash()`, `trailingslashit()`
User Functions	`wp_get_current_user()`, `get_current_user_id()`, `current_user_can()`
Option/Transient	`get_option()`, `update_option()`, `delete_option()`, `get_transient()`, `set_transient()`
Conditional Tags	`is_admin()`, `is_multisite()`, `is_user_logged_in()`, `is_singular()`
Classes	`WP_Error`, `WP_Widget`, `WP_List_Table`, `WP_Dependencies`, `IXR_Client`, `ftp_base`, `Translation_Entry`
Not modeled	Database connections, HTTP requests, filesystem writes, session state, full WordPress core bootstrap

C.5.2 Stubs and mocks

The stub file `wp_core.php` provides 82 function stubs, 7 class stubs, and 20 constant definitions. Table 2 summarizes the stubbed APIs and globals. The harness does *not* model full WordPress bootstrap; outcomes may therefore reflect harness-visible interface drift, missing initialization, or unmodeled dependencies rather than end-to-end behavior.

C.5.3 Error handling policy

The runner uses the following error-handling policy:

- **Fatal errors / Exceptions**: immediate failure (captured via try-catch)
- **Errors (E_ERROR, E_PARSE)**: immediate failure
- **Warnings / Notices / Deprecations**: captured by a custom error handler; diagnostics are treated as failures only when attributed to the target file (via substring match on `$errfile`)

C.6 Slice accounting

The harness is configured over a fixed slice of $N_{\mathrm{rt}} = 55$ target files. Some files have known isolation limitations (external dependencies or cross-file coupling) but remain included in both the slice and the denominator.

Table 3 Files with Known Isolation Limitations (Included in Slice)

File	Reason
`getid3.php`	Requires getID3 core library classes.
`module.audio.ac3.php`	Requires `getid3_handler` base class.
`class-ftp.php`	Cross-file dependency on FTP classes.
`class-pop3.php`	Depends on network/socket initialization.
`class-json.php`	Requires `Services_JSON` library.

C.7 Outcome taxonomy

A file is counted as **Pass** for a given codebase version if and only if it loads successfully without fatal errors. Include-time fatal errors, including parse errors, fatal runtime errors, and uncaught exceptions, are counted as failures.

Table 4 Outcome Categories for the Isolated Loadability Tripwire

Outcome	Interpretation
Both pass	Both original and migrated versions load successfully.
Fixed failure	The original fails to load, but the migrated version loads successfully.
New failure	The original loads successfully, but the migrated version fails to load.
Both fail	Both original and migrated versions fail to load.

For concision in the main text, we aggregate outcomes as follows: **Pass** = (Both pass + Fixed failure), **Regress** = (New failure), and **Other** = (Both fail).

C.8 Harness sanity checks

For transparency, we report absolute pass rates under identical harness conditions, computed over the full $N_{\mathrm{rt}} = 55$ slice. Because the harness does not boot WordPress and files are executed in isolation with stubs, these pass rates are harness-dependent and are *not* interpreted as integration or correctness measures. This tier is used primarily to surface *Regress* outcomes.

Table 5 Harness Pass Rates (Context Only; Not a Correctness Metric; $N_{\mathrm{rt}} = 55$)

Codebase Version	Passed	Pass Rate
Original snapshot	50	90.9%
Rector-migrated baseline	46	83.6%
Claude Sonnet 4	44	80.0%
GPT-5 Codex	43	78.2%
Gemini 2.5 Pro	40	72.7%
Llama 3.3 70B	38	69.1%
Gemini 2.5 Flash	33	60.0%

We therefore emphasize **Regress** counts in the main text and do not use absolute pass rates for model ranking.

Interpretation.

The original snapshot's non-ceiling pass rate (90.9%) reflects unavoidable isolation limitations (Table 3): five files fail to load because of external dependencies or deprecated PHP syntax (curly-brace array access) removed in PHP 8. Such cases are represented explicitly through the *Both fail / Other* outcome rather than excluded.

C.9 Failure exemplars

Representative harness-visible load-time failures from the test results include:

1. **Deprecated curly-brace array access syntax (original files).**
   Files: `class-ftp.php`, `class-pop3.php`, `getid3.php`, `module.audio.ac3.php`, `class-json.php`.
   These original files use `$var{0}` syntax, which was removed in PHP 8, or require external dependencies; as a result, 5 of 55 original files fail to load, yielding the 90.9% baseline pass rate.

2. **Callback transformation drops required parameters (Rector).**
   Files: `class.akismet-widget.php`, `user.php`, `file.php`.
   Rector converted closures to arrow functions and dropped required callback parameters, triggering fatal argument-count errors at callback invocation sites under the harness.
3. **Parse errors from malformed syntax (Gemini models).**
   Files: `Parser.php`, `File.php`.
   Gemini 2.5 Flash introduced malformed syntax, including unexpected tokens (e.g., "unexpected token ?" or "unexpected token ("), causing include-time parse failures.
4. **Modified include path points to a missing file (multiple models).**
   File: `translations.php`.
   Both Rector and Gemini 2.5 Pro modified `require_once` paths, causing "Failed opening required" errors when the referenced file does not exist.
5. **High regression rate coincides with low resolution.**
   Gemini 2.5 Flash showed 18 regressions (32.7%) with 66.6% rule resolution, whereas Rector showed 7 regressions (12.7%) with 98.7% resolution, illustrating the tripwire's sensitivity to incomplete or destabilizing migrations.

Summary.
The loadability tripwire detects include-time failures and harness-visible regressions, including parse errors, fatal errors, missing dependencies, and broken inheritance, that static contract discharge may not surface. Because all metrics are computed over a fixed $N_{\mathrm{rt}} = 55$ slice, unavoidable isolation limitations are represented through the *Both fail / Other* category rather than excluded. This tier provides a complementary regression signal, but it does not validate behavioral correctness or end-to-end functionality.

D Inferential Statistics Details

Table 6 Paired Wilcoxon details (LLMs).

Model A	Model B	p	Holm p	Δ_{med}
Gemini 2.5 Pro	GPT-5 Codex	3.49e-05	1.40e-04	0.000
Gemini 2.5 Pro	Gemini 2.5 Flash	9.55e-09	4.78e-08	0.196
Gemini 2.5 Pro	Claude Sonnet 4	1.78e-09	1.07e-08	0.196
Gemini 2.5 Pro	Llama 3.3 70B	3.12e-16	2.49e-15	0.391
GPT-5 Codex	Gemini 2.5 Flash	5.37e-02	1.61e-01	0.000
GPT-5 Codex	Claude Sonnet 4	8.57e-02	1.71e-01	0.000
GPT-5 Codex	Llama 3.3 70B	9.75e-11	6.83e-10	0.196

Note. Δ_{med} is the median paired difference (pp).
This appendix reports the exact paired-comparison outputs referenced in Section **??**.